INSTITUTO TÉCNICO RICALDONE · ESPECIALIDAD DE DESARROLLO DE SOFTWARE

Repaso para la Defensa Externa de PetCore

Sistema de gestión veterinaria: sistema web (HTML, CSS, Bootstrap, JavaScript sin frameworks), API (Java, Spring Boot, JPA/Hibernate), base de datos Oracle Cloud y app móvil Android/iOS con Capacitor.

Frontend → Vercel (petcore-seven.vercel.app)

API / Backend → Heroku (petcore-8afada45fabc.herokuapp.com)

Base de datos → Oracle Cloud, gestionada con SQL Developer

App móvil → Capacitor (Android / iOS)

Contenido

1. Rúbrica oficial de evaluación externa
2. Verificación técnica de último momento
3. Arquitectura del Backend (API Java / Spring Boot)
4. Arquitectura del Frontend y la App Móvil
5. Base de datos (Oracle Cloud)
6. Roles, autenticación, CRUD y alcance del Propietario en móvil
7. Conceptos clave explicados (32 temas)
8. Banco de preguntas por dificultad (53 preguntas)
9. Logística del equipo (6 integrantes, 4 días, 2 por turno)
10. Guion para el Jurado Externo
11. Guion para el Público

SECCIÓN 1

Rúbrica oficial de evaluación externa

- [ ] **1. Documentación del sistema.** Manual de usuario con imágenes + documentación técnica + diagramas.
- [ ] **2. Diseño de las interfaces web.** Responsive, fácil interacción, diseño profesional.
- [ ] **3. Diseño de las interfaces móviles.** Mismo criterio aplicado a la app móvil.
- [ ] **4. Funcionalidad del sistema web.** CRUD validado en Frontend y Backend.
- [ ] **5. Funcionalidad de la aplicación móvil.** Igual que el criterio 4, en la app móvil.
- [ ] **6. Validaciones e integridad de los datos.** Tipos correctos, verificado por la API.
- [ ] **7. Autenticación y roles de usuario.** Login + manejo de roles en Frontend y Backend.
- [ ] **8. Protección de endpoints y vistas.** Endpoints públicos/protegidos + bloqueo del botón "atrás".
- [ ] **9. Despliegue del Backend, sistema web y móvil.** CORS, URL pública, app en dispositivo físico.
- [ ] **10. Dominio.** Seguridad, lenguaje técnico, descripciones acertadas.

SECCIÓN 2 · CRITERIO 9

Verificación técnica de último momento

- [ ] **Vercel activo (dominio 1 y 2).** Confirmar que cargan el login, no un error.
- [ ] **API de Heroku despierta.** Cargar el login antes para evitar el "cold start".
- [ ] **Login de cada rol en producción.** Probar los 4 roles contra la URL pública.
- [ ] **App móvil instalada y probada** en el dispositivo físico.
- [ ] **Batería e internet del dispositivo móvil.**

SECCIÓN 3 · BACKEND · JAVA + SPRING BOOT

Arquitectura de la API de PetCore

Dependencias clave

DEPENDENCIA	USO
`spring-boot-starter-data-jpa`	Mapea las 51 tablas Oracle a entidades Java.
`spring-boot-starter-security` + `jjwt-*`	Autenticación/autorización, JWT en cookie.
`spring-security-crypto`	Usado junto a Argon2 para contraseñas.
`spring-boot-starter-websocket`	Endpoint `/ws` en tiempo real.
`google-api-client`	Verifica el ID Token de Google (OAuth2).
`cloudinary-http44`	Sube y sirve imágenes (fotos de mascotas/perfil).
`web-push` / `firebase-admin`	Push navegador (VAPID) y push nativo móvil (FCM).
`ojdbc11`	Driver JDBC de Oracle.
`lombok`	Genera getters/setters/constructores por anotación.

Arquitectura modular (51 módulos) y "deny by default"

Cada módulo repite 6 carpetas: `controller/ service/ repository/ entity/ dto/ exceptions/`. `SecurityConfig.java` termina siempre con:

```
auth.anyRequest().denyAll();
```

Cualquier endpoint nuevo sin regla explícita queda bloqueado automáticamente.

JWT + cookie httpOnly

`JWTUtils.java` firma con HMAC-SHA256. `JwtCookieAuthFilter.java` lee `authToken` (empleados) o `authTokenPropietario` (dueños). Sesión deslizante: renueva la cookie en cada request (TTL 900s).

Novedad: autoregistro del Propietario por DUI

`RegistroCuentaController.java` — un propietario ya cliente de la clínica puede crear su cuenta sin ayuda del personal:

PASO	ENDPOINT	QUÉ HACE
1	`POST /api/auth/registro/propietario/solicitar`	Envía código al correo ya registrado.
2	`POST /api/auth/registro/propietario/verificar`	Valida DUI + código.
3	`POST /api/auth/registro/propietario/crear`	Define usuario y contraseña.

> "Solicitar" responde siempre el mismo mensaje genérico (anti-enumeración de usuarios), igual que la recuperación

> de contraseña.

Triggers y reglas de negocio en la base de datos

- `TRG_HOSPITALIZACION_CHECK_CAPACITY`: rechaza un ingreso si ya se alcanzó la capacidad configurada.
- `TRG_PROD_INSERTAR_STOCK_MIN`: asigna stock mínimo/alerta automático a un producto nuevo.
- `TRG_DETALLE_VENTA_STOCK` + `PKG_KARDEX_CTX`: descuenta stock y registra el kardex al confirmar una venta, evitando duplicados con una bandera de control (`g_omitir`).

WebSockets y despliegue

WebSocket puro (sin STOMP), broadcast total vía `EventoWebSocketHandler.java`. Ping cada 25s evita que Heroku corte conexiones inactivas. Procfile: `java -jar target/API-0.0.1-SNAPSHOT.jar`, con GitHub Actions haciendo ping periódico contra el "cold start".

SECCIÓN 4 · FRONTEND · HTML/CSS/JS PURO + APP CAPACITOR

Arquitectura del Frontend y la App Móvil

Patrón Controller/Service en JS puro, páginas organizadas por rol. Sesión por **cookie httpOnly**, no localStorage — `fetch_con_credenciales.js` agrega `credentials:"include"` globalmente y reintenta ante 502/503/504.

```
window.addEventListener("pageshow", (e) => {
  if (e.persisted) location.reload();
});
```

Capacitor empaqueta el mismo frontend en un WebView nativo (`CapacitorCookies` habilitado para que la cookie de sesión funcione también ahí). **Novedad:** la app ahora tiene pantalla de "Crear cuenta" por DUI, llamando a los nuevos endpoints de `RegistroCuentaController`.

SECCIÓN 5 · BASE DE DATOS · ORACLE CLOUD

Modelo de datos de PetCore (51 tablas)

Catálogos y personal

`TBL_ROLES` `TBL_TURNOS` `TBL_EMPLEADOS` `TBL_CFG_SISTEMA`

Catálogos clínicos y mascotas

`TBL_ESPECIE` `TBL_RAZA` `TBL_PELAJE` **`TBL_PELAJE_ESPECIE (nueva)`** `TBL_PROPIETARIOS` `TBL_MASCOTAS`

`TBL_ALERGIAS` `TBL_ALERGIA_MASCOTA` `TBL_DIAGNOSTICOS` `TBL_DIAGNOSTICO_RAZA` `TBL_MASCOTA_ENFERMEDAD`

Citas y atención clínica

`TBL_SERVICIOS` `TBL_CITAS` `TBL_CITA_SERVICIO` `TBL_EXPEDIENTE` `TBL_DIAGNOSTICO_EXPEDIENTE`

`TBL_EXPEDIENTE_SERVICIO` `TBL_PRESCRIPCION` `TBL_DETALLE_PRESCRIPCION` `TBL_HOSPITALIZACION`

`TBL_EVOLUCION_CLINICA` `TBL_TAREAS_MEDICAS` `TBL_CARTILLAS` `TBL_ANTIPARASITARIO` `TBL_HISTORIAL_TRATAMIENTO`

`TBL_CUIDADOS` `TBL_POSOLOGIA_FARMACO`

Inventario, compras y ventas

`TBL_PROVEEDOR` `TBL_TIPO_PRODUCTO` `TBL_MARCA` `TBL_PRODUCTO` `TBL_COMPRAS` `TBL_DETALLE_COMPRA`

`TBL_SOLICITUDES_BODEGA` `TBL_DETALLE_SOLICITUD` `TBL_VENTAS` `TBL_DETALLE_VENTA` `TBL_MOVIMIENTO_INVENTARIO`

Comunicación y notificaciones

`TBL_ANUNCIOS` `TBL_ANUNCIO_REACCIONES` `TBL_ANUNCIO_COMENTARIOS` `TBL_NOTIFICACIONES`

`TBL_NOTIFICACION_DESTINATARIO` `TBL_SUSCRIPCIONES_PUSH` `TBL_PUSH_TOKEN_FCM`

Recursos humanos

`TBL_ASISTENCIA_EMPLEADOS` `TBL_DIAS_FERIADOS`

Cambio reciente: mascotas, la especie ya no depende de la raza

- `MAS_ESP_ID` → ahora obligatoria, FK directa a Especie.
- `MAS_RAZ_ID` y `MAS_PEL_ID` → ahora opcionales.
- FK compuesta `(MAS_RAZ_ID, MAS_ESP_ID) → TBL_RAZA(RAZ_ID, RAZ_ESP_ID)`.
- `TBL_RAZA.RAZ_SIN_PELAJE`: marca razas sin pelo (Sphynx, Xoloitzcuintle...).
- `TBL_PELAJE_ESPECIE` (nueva): qué pelajes aplican a cada especie.

Flujo clínico completo

```
PROPIETARIO → MASCOTA → CITA → CITA_SERVICIO → EXPEDIENTE → DIAGNOSTICO_EXPEDIENTE → PRESCRIPCION
→ DETALLE_PRESCRIPCION → MOVIMIENTO_INVENTARIO
```

SECCIÓN 6 · ROLES, AUTENTICACIÓN Y CRUD

Roles, autenticación y funcionalidad CRUD

ROL	ENFOQUE
Administrador	Control total: personal, inventario, compras, reportes, configuración.
Veterinario	Atención clínica: expedientes, diagnósticos, hospitalización.
Recepcionista	Front desk: propietarios/mascotas, citas, sala de espera.
Farmacéutico	Farmacia: POS, stock, ventas.
Propietario	App móvil: ver mascotas, agendar citas, historial de solo lectura.

Alcance CRUD real del Propietario en móvil

MÓDULO	SÍ PUEDE	NO PUEDE (POR QUÉ)
Mascotas	Ver propias; actualizar foto	Crear/editar datos clínicos — requiere verificación presencial.
Citas	Ver; crear nueva	Editar/cancelar — lo gestiona el personal.
Expediente	Ver (solo lectura)	Editar diagnósticos o notas clínicas.
Cuenta propia	Autoregistro por DUI, autoeditar, cambiar contraseña	Editar perfil de otro propietario.

SECCIÓN 7 · 32 CONCEPTOS CLAVE

Conceptos clave, explicados con el código real

¿Qué es CORS?

Mecanismo del navegador que decide si un sitio puede llamar a una API en otro dominio. PetCore usa lista blanca + `allowCredentials=true`.

¿Qué son los WebSockets?

Canal bidireccional persistente. PetCore usa WebSocket puro con broadcast total.

¿Qué guardan en localStorage?

Solo preferencias de interfaz. El token vive en cookie httpOnly, inaccesible a JS.

¿Cómo funciona el JWT en cookie?

Firmado, dentro de cookie httpOnly. Sesión deslizante cada 15 min.

¿Cómo evitan CSRF con cookies?

`SameSite` + `Secure` + CORS + API JSON pura.

¿Cómo bloquean el botón "Atrás"?

Evento `pageshow` con `event.persisted` fuerza recarga.

¿Cómo protegen tantos endpoints?

"Deny by default": `denyAll()` al final de la cadena.

¿Login con Google?

`GoogleTokenVerifier` verifica el ID Token con los servidores de Google.

¿Es PWA? ¿Service Worker?

Parcial: `sw.js` solo maneja Web Push.

¿Frontend o backend valida?

Ambos — la garantía real está en el backend.

¿Por qué Capacitor y no nativo?

Reutiliza el 100% del frontend web en un WebView nativo.

¿Qué es REST?

Recursos + métodos HTTP, stateless.

¿Qué es un DTO?

Objeto de transferencia separado de la entidad JPA.

¿ORM y problema N+1?

Hibernate; N+1 mitigado con `batch_fetch_size=50`.

¿Qué es Maven?

Gestor de dependencias y build de Java.

¿Qué es "stateless"?

El servidor no guarda sesión — clave ante reinicios de Heroku.

¿Qué es idempotencia?

Repetir la operación da el mismo resultado — GET/DELETE la necesitan.

¿HTTPS/TLS y cookies?

Cifra el tránsito; cookie con atributo `Secure`.

¿Qué es un índice (IX_)?

Estructura para encontrar filas rápido sin recorrer toda la tabla.

¿Transacción y ACID?

Todo o nada; Atomicidad, Consistencia, Aislamiento, Durabilidad.

¿Argon2 vs bcrypt?

Argon2 es costoso también en memoria, más resistente a GPUs/ASICs.

¿Qué es VAPID?

Identifica al servidor ante los servicios push del navegador.

¿Por qué responde siempre el mismo mensaje al solicitar código?

Evita **enumeración de usuarios** — mismo patrón en recuperación de contraseña y autoregistro por DUI.

¿Qué es un trigger y dónde los usa PetCore?

`TRG_HOSPITALIZACION_CHECK_CAPACITY` (límite de camas), `TRG_PROD_INSERTAR_STOCK_MIN` (stock mínimo automático), `TRG_DETALLE_VENTA_STOCK` (descuenta stock y registra kardex al vender) — reglas que corren en la base de datos sin importar el origen del cambio.

¿Qué es el "kardex" y cómo evita duplicados?

Historial de movimientos de inventario. `PKG_KARDEX_CTX` usa una bandera (`g_omitir`) para que un trigger de venta y uno genérico no registren el mismo movimiento dos veces.

¿Inyección de dependencias / @RequiredArgsConstructor?

Spring inyecta las dependencias por constructor; Lombok genera ese constructor a partir de los campos `final`.

¿FetchType.LAZY?

Carga la relación solo si el código la usa — evita arrastrar en cascada datos relacionados innecesarios.

¿Pool de conexiones (HikariCP)?

Reutiliza conexiones abiertas; limitado a 8 por los límites de Oracle Cloud y Heroku.

¿Por qué Cloudinary y no guardar fotos en la BD?

BLOB infla la base de datos; el disco de Heroku es efímero. Cloudinary guarda la imagen y PetCore solo la URL.

¿Por qué SweetAlert2 y no alert()/confirm()?

Los diálogos nativos bloquean la página y no se pueden personalizar; SweetAlert2 sí, con el estilo de PetCore.

¿Qué es OAuth 2.0?

PetCore nunca ve la contraseña de Google — solo recibe y verifica un token firmado.

SECCIÓN 8 · 53 PREGUNTAS POR DIFICULTAD

Banco de preguntas para la defensa externa

1. Documentación

FÁCIL **¿Qué documentación tiene el proyecto?**

Manual de usuario + técnica + diagramas.

DIFÍCIL **¿Qué leería primero un desarrollador nuevo?**

Instalación → arquitectura → modelo ER → un módulo de ejemplo.

2-3. Diseño de interfaces

FÁCIL **¿Qué es responsive?**

Se adapta al tamaño de pantalla.

DIFÍCIL **¿Consistencia visual sin React?**

Variables CSS + controller común de estructura.

4-5. CRUD web y móvil

FÁCIL **¿Qué es CRUD?**

Crear, Leer, Actualizar, Eliminar.

INTERMEDIA **¿Quién descuenta el stock al vender: frontend, backend o BD?**

La base de datos, vía trigger (`TRG_DETALLE_VENTA_STOCK`).

DIFÍCIL **¿Dos usuarios editando el mismo registro?**

"Último que guarda, gana" — mejora: `@Version` de JPA.

DIFÍCIL **¿Eliminar mascota con expedientes?**

FK lo impide → 409. Por eso se usa baja lógica.

6. Validaciones

FÁCIL **Ejemplo de campo validado**

DUI: regex + CHECK en Oracle.

INTERMEDIA **¿Por qué mascota sin raza/pelaje?**

Especie es el único dato obligatorio; raza/pelaje ahora opcionales.

DIFÍCIL **¿Raza de otra especie asignada por error?**

FK compuesta `(RAZ_ID, RAZ_ESP_ID)` lo impide en la BD.

DIFÍCIL **¿Hospitalizar sin camas disponibles?**

Trigger `TRG_HOSPITALIZACION_CHECK_CAPACITY` rechaza el ingreso.

7. Autenticación y roles

FÁCIL **¿Cuántos roles hay?**

Cinco: Admin, Veterinario, Recepcionista, Farmacéutico, Propietario.

INTERMEDIA **¿Cómo crea cuenta un propietario nuevo?**

Por DUI: solicitar código → verificar → crear usuario/contraseña.

INTERMEDIA **¿PetCore ve la contraseña de Google?**

Nunca — OAuth2 solo entrega un token verificado.

DIFÍCIL **¿Por qué el autoregistro no revela si el DUI existe?**

Anti-enumeración de usuarios.

8. Protección de endpoints

FÁCIL **¿URL sin sesión?**

Redirige al login automáticamente.

INTERMEDIA **¿401 vs 403?**

401: no sé quién eres. 403: sé quién eres, sin permiso.

DIFÍCIL **¿Por qué /ws es público?**

Debilidad reconocida: el handshake no valida JWT todavía.

9. Despliegue

FÁCIL **¿Qué es un CDN?**

Red de servidores que sirve estáticos cerca del usuario — lo usa Vercel.

INTERMEDIA **¿Cold start de Heroku?**

Dyno duerme sin tráfico; mitigado con reintentos + ping de GitHub Actions.

DIFÍCIL **¿Si Cloudinary se cae?**

Fallan subidas nuevas; el resto del sistema sigue funcionando.

10. Dominio

FÁCIL **¿Por qué separar controller/service?**

Patrón Service Layer: testable y reutilizable.

FÁCIL **¿Qué es Lombok?**

Genera getters/setters/constructores por anotación.

DIFÍCIL **¿Por qué Oracle y no MySQL?**

Requisito institucional + `GenerationType.IDENTITY` compatible.

DIFÍCIL **¿Escalar a 50 clínicas?**

Multi-tenancy, balanceo de carga, particionamiento.

SECCIÓN 9

Logística: 6 integrantes, 4 días, 2 por turno

DÍA	MATUTINO	VESPERTINO
Día 1	_____ · _____	_____ · _____
Día 2	_____ · _____	_____ · _____
Día 3	_____ · _____	_____ · _____
Día 4	_____ · _____	_____ · _____

> Cualquier pareja debe poder ejecutar ambos guiones completos.

SECCIÓN 10

⚖️ Guion para el Jurado Externo

1. Apertura ~2 min

Presentación del equipo y los 5 roles (4 de personal + Propietario).

2. Documentación ~2 min

Manual de usuario, técnica, diagrama de casos de uso.

3. Demo web: los 4 roles de personal ~14 min

Administrador (CRUD) → Veterinario (expediente) → Recepcionista (mascota + cita) → Farmacéutico (POS).

4. Protección de endpoints ~5 min

Público, 401, 403, botón "Atrás".

5. Demo API (Heroku) ~5 min

200 vs 401, ApiResponse, CORS.

6. Demo base de datos ~5 min

SQL Developer, modelo, constraint.

7. Demo app móvil: Propietario ~7 min

Login/crear cuenta por DUI, mascotas, cita, push.

8. Arquitectura y dominio técnico Sin límite

51 módulos, JWT, deny-by-default. Preguntas libres.

9. Cierre ~1 min

Agradecer.

SECCIÓN 11

📣 Guion para el Público (resumido)

1. Gancho ~20 seg

"¿Tienen mascota? Les mostramos PetCore en 3 minutos."

2. El problema ~30 seg

Centraliza citas, historial, inventario — pantalla por tipo de empleado + app para el dueño.

3. Demo ~2 min

Cita, expediente con foto, app móvil, POS si hay interés.

4. Lo especial ~30 seg

"Lo construimos nosotros y está en producción ahora mismo."

5. Cierre ~20 seg

"¡Gracias por pasar!"